

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
  <title>Quantum Core Processor - Current
I/O Detector</title>
  <!-- Lucide Icons -->
  <script src="https://unpkg.com/
lucide@latest"></script>
  <style>
    :root {
      --bg-dark: #030712;
      --panel-bg: rgba(15, 23, 42, 0.75);
      --border-glow: rgba(56, 189, 248,
0.25);
      --text-main: #f8fafc;
      --text-dim: #94a3b8;
```

```
/* Spectrum Colors */  
--q-green: #10b981;  
--q-blue: #3b82f6;  
--q-red: #ef4444;  
--q-pink: #ec4899;  
--q-white: #ffffff;  
--q-yellow: #eab308;  
--q-orange: #f97316;  
--q-cyan: #06b6d4;  
}
```

```
* {  
  box-sizing: border-box;  
  margin: 0;  
  padding: 0;  
  font-family: 'Segoe UI', -apple-  
system, BlinkMacSystemFont, Roboto,  
sans-serif;  
}
```

```
body {
```

```
background-color: var(--bg-dark);
color: var(--text-main);
min-height: 100vh;
display: flex;
flex-direction: column;
align-items: center;
padding: 20px;
overflow-x: hidden;
}
```

```
/* App Container */
.app-container {
  width: 100%;
  max-width: 1400px;
  display: flex;
  flex-direction: column;
  gap: 20px;
}
```

```
/* Glassmorphism Panel Base */
.panel {
```

```
background: var(--panel-bg);
border: 1px solid var(--border-glow);
border-radius: 16px;
padding: 24px;
backdrop-filter: blur(12px);
box-shadow: 0 8px 32px 0 rgba(0, 0,
0, 0.6);
position: relative;
}
```

```
/* Header Navigation */
```

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 16px;
}
```

```
.header-title {
  font-size: 1.3rem;
```



```
font-weight: 700;
color: #38bdf8;
display: flex;
align-items: center;
gap: 12px;
letter-spacing: 1px;
text-transform: uppercase;
}
```

```
.cpu-glow-icon {
  color: var(--q-cyan);
  animation: cpuPulse 2s infinite
alternate;
}
```

```
@keyframes cpuPulse {
  0% { filter: drop-shadow(0 0 2px
var(--q-cyan)); transform: scale(1); }
  100% { filter: drop-shadow(0 0 12px
var(--q-pink)); transform: scale(1.1); }
}
```

```
.nav-tabs {  
  display: flex;  
  gap: 10px;  
  background: rgba(0, 0, 0, 0.4);  
  padding: 6px;  
  border-radius: 10px;  
  border: 1px solid var(--border-glow);  
}
```

```
.nav-btn {  
  background: transparent;  
  border: none;  
  color: var(--text-dim);  
  padding: 8px 18px;  
  border-radius: 6px;  
  cursor: pointer;  
  font-weight: 600;  
  font-size: 0.9rem;  
  display: flex;  
  align-items: center;
```

```
    gap: 8px;
    transition: all 0.3s ease;
}
```

```
.nav-btn:hover, .nav-btn.active {
    background: var(--q-cyan);
    color: #000;
    box-shadow: 0 0 12px var(--q-cyan);
}
```

```
/* Layout Grid */
.main-dashboard {
    display: grid;
    grid-template-columns: 1fr;
    gap: 20px;
}
```

```
@media (min-width: 1024px) {
    .main-dashboard {
        grid-template-columns: 320px 1fr
        340px;
```

```
}  
}
```

```
/* Views management */  
.tab-view { display: none; }  
.tab-view.active { display: block; }
```

```
.panel-header-title {  
  font-size: 1rem;  
  text-transform: uppercase;  
  letter-spacing: 1px;  
  color: var(--q-cyan);  
  display: flex;  
  align-items: center;  
  gap: 8px;  
  margin-bottom: 16px;  
  border-bottom: 1px solid var(--  
border-glow);  
  padding-bottom: 8px;  
}
```

```
/* Central Glowing Colorful Grid  
Visualizer */
```

```
.visualizer-container {  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  justify-content: center;  
  position: relative;  
  min-height: 320px;  
  margin-bottom: 20px;  
}
```

```
.quantum-chip {  
  width: 280px;  
  height: 280px;  
  background: #080c14;  
  border: 2px solid rgba(255, 255,  
255, 0.15);  
  border-radius: 12px;  
  position: relative;  
  display: flex;
```

```
    justify-content: center;
    align-items: center;
    box-shadow: 0 0 35px rgba(56, 189,
248, 0.2);
}
```

```
.chip-grid {
    width: 230px;
    height: 230px;
    display: grid;
    grid-template-columns: repeat(10,
1fr);
    grid-template-rows: repeat(10, 1fr);
    gap: 3px;
    padding: 4px;
    background: rgba(0, 0, 0, 0.8);
    border-radius: 6px;
    position: relative;
    z-index: 2;
}
```

```
.grid-cell {  
    background-color: #1e293b;  
    border-radius: 1px;  
    transition: background-color 0.2s  
ease, box-shadow 0.2s ease;  
}
```

```
/* Spectrum Colors Glow */
```

```
.grid-cell.active-green { background-  
color: var(--q-green); box-shadow: 0 0 6px  
var(--q-green); }
```

```
.grid-cell.active-blue { background-  
color: var(--q-blue); box-shadow: 0 0 6px  
var(--q-blue); }
```

```
.grid-cell.active-red { background-  
color: var(--q-red); box-shadow: 0 0 6px  
var(--q-red); }
```

```
.grid-cell.active-pink { background-  
color: var(--q-pink); box-shadow: 0 0 6px  
var(--q-pink); }
```

```
.grid-cell.active-white { background-
```

```
color: var(--q-white); box-shadow: 0 0 6px  
var(--q-white); }
```

```
.grid-cell.active-yellow { background-  
color: var(--q-yellow); box-shadow: 0 0 6px  
var(--q-yellow); }
```

```
.grid-cell.active-orange { background-  
color: var(--q-orange); box-shadow: 0 0 6px  
var(--q-orange); }
```

```
.core-glow {  
    position: absolute;  
    width: 200px;  
    height: 200px;  
    background: radial-gradient(circle,  
    rgba(236,72,153,0.3) 0%,  
    rgba(234,179,8,0.2) 30%,  
    rgba(59,130,246,0.3) 60%,  
    rgba(16,185,129,0.2) 90%, transparent  
    100%);  
    border-radius: 50%;  
    filter: blur(25px);
```



```
        animation:
continuousSpectrumGlow 4s ease-in-out
infinite alternate;
        z-index: 1;
    }
```

```
    @keyframes
continuousSpectrumGlow {
        0% { transform: scale(0.9); filter:
blur(20px) hue-rotate(0deg); opacity: 0.8; }
        100% { transform: scale(1.25); filter:
blur(35px) hue-rotate(360deg); opacity: 1; }
    }
```

```
/* Qubit Scale Bar (61,000 Qubits) */
.qubit-tracker { margin-bottom: 20px; }
.progress-bar-bg {
    width: 100%;
    height: 12px;
    background: rgba(255, 255, 255,
0.05);
```

```
border-radius: 6px;
overflow: hidden;
border: 1px solid var(--border-glow);
margin-top: 6px;
}

.progress-bar-fill {
height: 100%;
width: 0%;
background: linear-gradient(90deg,
var(--q-green), var(--q-blue), var(--q-pink),
var(--q-yellow), var(--q-orange));
border-radius: 6px;
transition: width 0.15s ease-out;
box-shadow: 0 0 10px var(--q-pink);
}
```

```
/* Silicon Canvas View */
.silicon-canvas-container {
width: 100%;
height: 260px;
background: #020617;
```

```
border: 1px solid var(--border-glow);
border-radius: 10px;
position: relative;
overflow: hidden;
display: flex;
align-items: center;
justify-content: center;
}
```

```
.silicon-canvas-svg { width: 100%;
height: 100%; }
```

```
/* Controls Section */
.control-group {
  display: flex;
  flex-direction: column;
  gap: 12px;
  margin-bottom: 16px;
}
```

```
.control-label {
```

```
font-size: 0.85rem;  
color: var(--text-dim);  
display: flex;  
justify-content: space-between;  
}
```

```
.slider-input {  
  width: 100%;  
  accent-color: var(--q-cyan);  
  cursor: pointer;  
}
```

```
.btn-toggle {  
  background: rgba(255, 255, 255,  
0.05);  
  border: 1px solid var(--border-glow);  
  color: var(--text-main);  
  padding: 10px;  
  border-radius: 8px;  
  cursor: pointer;  
  font-weight: bold;
```

```
display: flex;
justify-content: space-between;
align-items: center;
transition: 0.2s;
}
```

```
.btn-toggle.active {
  background: rgba(16, 185, 129, 0.2);
  border-color: var(--q-green);
  color: var(--q-green);
}
```

```
/* Metrics List */
.feature-grid {
  display: grid;
  grid-template-columns: repeat(auto-
fit, minmax(130px, 1fr));
  gap: 12px;
}
```

```
.feature-card {
```

```
background: rgba(255, 255, 255,
0.03);
border: 1px solid var(--border-glow);
padding: 12px;
border-radius: 8px;
}
```

```
.feature-title { font-size: 0.75rem;
color: var(--text-dim); }
```

```
.feature-val { font-size: 1.1rem; font-
weight: bold; font-family: monospace;
margin-top: 4px; }
```

```
/* Logic Gates / Learn Section */
.gate-card-grid {
display: grid;
grid-template-columns: repeat(auto-
fit, minmax(220px, 1fr));
gap: 16px;
}
```

```
.gate-card {  
    background: rgba(255, 255, 255,  
0.03);  
    border: 1px solid var(--border-glow);  
    padding: 16px;  
    border-radius: 10px;  
}
```

```
.gate-title { color: var(--q-cyan); font-  
weight: bold; margin-bottom: 6px; display:  
flex; align-items: center; gap: 6px;}
```

```
/* Oscilloscope Simulation View */  
.oscilloscope {  
    width: 100%;  
    height: 200px;  
    background: #000;  
    border: 1px solid var(--q-green);  
    border-radius: 8px;  
    position: relative;  
}
```

```
/* CPU & Storage Functional Spec  
Cards */
```

```
.spec-list {  
  display: flex;  
  flex-direction: column;  
  gap: 10px;  
  font-size: 0.85rem;  
}
```

```
.spec-item {  
  background: rgba(255, 255, 255,  
0.02);  
  padding: 10px;  
  border-radius: 6px;  
  border-left: 3px solid var(--q-cyan);  
}
```

```
.spec-item strong { color: var(--text-  
main); display: block; margin-bottom: 2px;}
```



```
</style>
</head>
<body>

  <div class="app-container">
    <!-- Top Navbar with Title & Navigation
-->
    <header class="panel navbar">
      <div class="header-title">
        <i data-lucide="cpu" class="cpu-
glow-icon"></i>
        Quantum Core Processor I/O
Detector
      </div>
      <nav class="nav-tabs">
        <button class="nav-btn active"
onclick="switchTab('build')">
          <i data-lucide="layout"></i>
          BUILD
        </button>
        <button class="nav-btn"
```

```
onclick="switchTab('simulate')">
    <i data-lucide="activity"></i>
SIMULATE
</button>
<button class="nav-btn"
onclick="switchTab('learn')">
    <i data-lucide="book-open"></i>
LEARN
</button>
</nav>
</header>

<!-- Dynamic Main Views -->
<!-- VIEW 1: BUILD / SANDBOX MODE --
>
<div id="view-build" class="tab-view
active">
    <div class="main-dashboard">

        <!-- Left Panel: Interactive
Controls & Capabilities -->
```

```
<div class="panel">
  <div class="panel-header-title">
    <i data-lucide="sliders-
horizontal"></i>
    INTERACTIVE CONTROLS
  </div>

  <div class="control-group">
    <div class="control-label">
      <span>Clock Frequency</
span>
      <span id="freqVal">4.8
GHz</span>
    </div>
    <input type="range"
class="slider-input" min="1" max="10"
value="5" id="freqSlider"
oninput="updateControls()">
  </div>

  <div class="control-group">
```

```
<div class="control-label">
```

```
<span>Target Qubits
```

```
Scale</span>
```

```
<span
```

```
id="qubitTargetVal">61,000 Qubits</span>
```

```
</div>
```

```
<input type="range"
```

```
class="slider-input" min="1000"
```

```
max="61000" step="1000" value="61000"
```

```
id="qubitSlider"
```

```
oninput="updateControls()">
```

```
</div>
```

```
<div class="control-group">
```

```
<button class="btn-toggle
```

```
active" id="inputA"
```

```
onclick="toggleSignal('A')">
```

```
<span>Signal Input A
```

```
[Transistor Switch]</span>
```

```
<span id="stateA">BINARY
```

```
1 (ON)</span>
```

```
</button>
```

```
<button class="btn-toggle"
```

```
id="inputB" onclick="toggleSignal('B')">
```

```
<span>Signal Input B
```

```
[Transistor Switch]</span>
```

```
<span id="stateB">BINARY
```

```
0 (OFF)</span>
```

```
</button>
```

```
</div>
```

```
<div class="panel-header-title"
```

```
style="margin-top: 20px;">
```

```
<i data-lucide="hard-drive"></i>
```

```
i>
```

```
STORAGE & CONTROLS SPEC
```

```
</div>
```

```
<div class="spec-list">
```

```
<div class="spec-item">
```

```
<strong> ⚡ Logic &
```

```
Computation:</strong>
```

```
Executes mathematical
```

operations & powers CPUs and GPUs.

</div>

<div class="spec-item">

 Temporary

Memory:

Retains working data

(RAM).

</div>

<div class="spec-item">

 Permanent

Storage:

Saves non-volatile

permanent files (NAND Flash).

</div>

<div class="spec-item">

 Startup ROM:</

strong>

Holds fundamental system

startup instructions.

</div>

</div>

```
</div>
```

```
<!-- Center Panel: Live Signal Flow  
Engine & Spectrum Core -->
```

```
<div class="panel">
```

```
<div class="panel-header-title">
```

```
<i data-lucide="zap"></i>
```

```
LIVE SIGNAL FLOW ENGINE
```

```
</div>
```

```
<!-- Spectrum Glowing Core  
Matrix Visualizer -->
```

```
<div class="visualizer-  
container">
```

```
<div class="core-glow"></div>
```

```
<div class="quantum-chip">
```

```
<div class="chip-grid"
```

```
id="chipGrid"></div>
```

```
</div>
```

```
</div>
```

```
<!-- Live Semiconductor Silicon  
Canvas View -->
```

```
<div class="panel-header-title"  
style="font-size: 0.85rem;">
```

```
<i data-lucide="binary"></i>  
SILICON CHIP DIGITAL SIGNAL CANVAS  
</div>
```

```
<div class="silicon-canvas-  
container">
```

```
<svg class="silicon-canvas-  
svg" viewBox="0 0 500 200">
```

```
<!-- Background Circuit  
Traces -->
```

```
<path d="M 50 50 L 180 50  
L 250 100 L 450 100" stroke="#1e293b"  
stroke-width="4" fill="none"/>
```

```
<path d="M 50 150 L 180  
150 L 250 100 L 450 100"  
stroke="#1e293b" stroke-width="4"  
fill="none"/>
```


<!-- Dynamic Signal Flow

Waves -->

<path id="wireA" d="M 50
50 L 180 50 L 250 100" stroke="var(--q-
cyan)" stroke-width="3" fill="none" stroke-
dasharray="8 4"/>

<path id="wireB" d="M 50
150 L 180 150 L 250 100"
stroke="#334155" stroke-width="3"
fill="none"/>

<path id="wireOut" d="M
250 100 L 450 100" stroke="var(--q-green)"
stroke-width="4" fill="none" stroke-
dasharray="10 5"/>

<!-- Gate Box Node -->

<rect x="210" y="75"
width="80" height="50" rx="8"
fill="#0f172a" stroke="var(--q-cyan)" stroke-
width="2"/>

<text x="250" y="105"

```
fill="#fff" font-size="12" font-weight="bold"
text-anchor="middle"
id="gateTextNode">XOR GATE</text>
```

```
<!-- Electron Flow
```

```
Visualizer Particles -->
```

```
<circle cx="50" cy="50"
r="4" fill="var(--q-cyan)">
```

```
<animate
attributeName="cx" values="50;180;250"
dur="1.5s" repeatCount="indefinite"/>
```

```
<animate
attributeName="cy" values="50;50;100"
dur="1.5s" repeatCount="indefinite"/>
```

```
</circle>
```

```
<circle cx="250" cy="100"
r="5" fill="var(--q-green)">
```

```
<animate
attributeName="cx" values="250;450"
dur="1s" repeatCount="indefinite"/>
```

```
</circle>
```

```
        </svg>
    </div>
</div>

<!-- Right Panel: Detector Metrics
& Real-time Readouts -->
<div class="panel">
    <div class="panel-header-title">
        <i data-lucide="gauge"></i>
        CURRENT I/O DETECTOR
    </div>

    <!-- Qubit Detector Tracker -->
    <div class="qubit-tracker">
        <div class="control-label">
            <span>Measured Qubits
Capacity</span>
            <span style="color: var(--q-
cyan); font-weight: bold;"
id="qubitCounter">0 / 61,000</span>
        </div>
```

```
        <div class="progress-bar-bg">
            <div class="progress-bar-
fill" id="qubitProgress"></div>
        </div>
    </div>
```

```
    <!-- Metrics Grid -->
    <div class="feature-grid"
style="margin-bottom: 20px;">
        <div class="feature-card">
            <div class="feature-
title">Logic State</div>
            <div class="feature-val"
style="color: var(--q-green);"
id="logicOutput">HIGH (1)</div>
        </div>
        <div class="feature-card">
            <div class="feature-
title">Signal Flow</div>
            <div class="feature-val"
id="signalFlowVal">Active</div>
```

```
</div>
```

```
<div class="feature-card">
```

```
<div class="feature-
```

```
title">Signal Flow</div>
```

```
<div class="feature-val"
```

```
id="signalFlowVal">Active</div>
```

```
</div>
```

```
<div class="feature-card">
```

```
<div class="feature-
```

```
title">Core Temp</div>
```

```
<div class="feature-val"
```

```
style="color: var(--q-pink);">12.4 mK</div>
```

```
</div>
```

```
<div class="feature-card">
```

```
<div class="feature-
```

```
title">Fidelity</div>
```

```
<div class="feature-val"
```

```
style="color: var(--q-yellow);">99.98%</div>
```

```
</div>
```

```
</div>
```

```
<div class="panel-header-title" style="font-size: 0.85rem;">
```

```
<i data-lucide="cpu"></i>
```

CPU SIMULATION CORE

```
</div>
```

```
<div class="spec-list">
```

```
<div class="spec-item">
```

```
<strong><img alt="Bar chart icon" data-bbox="228 338 282 378"/> Core Digital Functioning:</strong>
```

Turns electrical currents on and off to represent binary 1s and 0s.

```
</div>
```

```
<div class="spec-item">
```

```
<strong><img alt="Signal switching icon" data-bbox="555 608 610 648"/> Signal Switching:</strong>
```

Directs complex system control flows across multi-threaded execution pipelines.

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
</div>
```

```
<!-- VIEW 2: SIMULATE MODE
(Oscilloscope Waveform) -->
```

```
<div id="view-simulate" class="tab-
view">
```

```
<div class="panel">
```

```
<div class="panel-header-title">
```

```
<i data-lucide="activity"></i>
```

```
DYNAMIC OSCILLOSCOPE
```

```
WAVEFORM MONITOR
```

```
</div>
```

```
<p style="color: var(--text-dim);
margin-bottom: 16px; font-size: 0.9rem;">
```

```
Real-time signal voltage
visualization across high-frequency
quantum transistor registers.
```

```
</p>
```

```
<div class="oscilloscope">
```

```
        <canvas id="scopeCanvas"
style="width: 100%; height: 100%;"></
canvas>

    </div>
</div>
</div>

<!-- VIEW 3: LEARN MODE (Digital Logic &
Quantum Theory) -->
    <div id="view-learn" class="tab-view">
        <div class="panel">
            <div class="panel-header-title">
                <i data-lucide="book-open"></i>
                DIGITAL LOGIC GATES &
SEMICONDUCTOR THEORY
            </div>
            <div class="gate-card-grid">
                <div class="gate-card">
                    <div class="gate-title"><i data-
lucide="git-commit"></i> AND Gate</div>
                    <p style="font-size: 0.85rem;
```


color: var(--text-dim);">Outputs 1 ONLY
when all input signals are turned ON (High
current flow).</p>

</div>

<div class="gate-card">

<div class="gate-title"><i data-
lucide="git-pull-request"></i> OR Gate</
div>

<p style="font-size: 0.85rem;
color: var(--text-dim);">Outputs 1 if AT
LEAST ONE input signal is ON.</p>

</div>

<div class="gate-card">

<div class="gate-title"><i data-
lucide="split"></i> XOR Gate</div>

<p style="font-size: 0.85rem;
color: var(--text-dim);">Outputs
1 if AT LEAST ONE input signal is ON.</p>

</div>

<div class="gate-card">

<div class="gate-title"><i data-

lucide="split"></i> XOR Gate</div>

<p style="font-size: 0.85rem; color: var(--text-dim);">Outputs 1 if inputs are DIFFERENT.

Fundamental for binary arithmetic adders.</p>

</div>

<div class="gate-card">

<div class="gate-title"><i data-lucide="shield-alert"></i> NOT Gate</div>

<p style="font-size: 0.85rem; color: var(--text-dim);">Inverts the incoming signal state (0 to 1, or 1 to 0).</p>

</div>

</div>

</div>

</div>

</div>

```
<script>
    // Initialize Icons
    lucide.createIcons();

    // TAB SWITCHING LOGIC
    function switchTab(tabName) {
        document.querySelectorAll('.tab-
view').forEach(el =>
el.classList.remove('active'));
        document.querySelectorAll('.nav-
btn').forEach(el =>
el.classList.remove('active'));

        document.getElementById(`view-$
{tabName}`).classList.add('active');

event.currentTarget.classList.add('active');

        if(tabName === 'simulate') {
            initOscilloscope();
        }
    }
}
```

```
}
```

```
// GENERATE 100-CELL SPECTRUM  
QUANTUM GRID (Central Glowing Feature)  
    const chipGrid =  
document.getElementById('chipGrid');  
    const gridCells = [];  
    const colors = ['active-green', 'active-  
blue', 'active-red', 'active-pink', 'active-white',  
'active-yellow', 'active-orange'];  
  
    for(let i = 0; i < 100; i++) {  
        const cell =  
document.createElement('div');  
        cell.className = 'grid-cell';  
        chipGrid.appendChild(cell);  
        gridCells.push(cell);  
    }  
  
function animateSpectrumGrid() {  
    gridCells.forEach(cell => {
```

```
        if (Math.random() > 0.5) {
            cell.className = 'grid-cell';
            if (Math.random() > 0.3) {
                const randomColor =
colors[Math.floor(Math.random() *
colors.length)];

cell.classList.add(randomColor);
            }
        }
    });
}
```

```
setInterval(animateSpectrumGrid, 180);
```

```
// SIGNAL & CONTROL STATES
```

```
let signalA = true;
```

```
let signalB = false;
```

```
let maxQubitsTarget = 61000;
```

```
let currentQubits = 0;
```

```
function toggleSignal(input) {  
    if(input === 'A') {  
        signalA = !signalA;  
        const btnA =  
document.getElementById('inputA');  
        btnA.classList.toggle('active',  
signalA);  
  
document.getElementById('stateA').innerTe  
xt = signalA ? 'BINARY 1 (ON)' :  
'BINARY 0 (OFF)';  
    } else {  
        signalB = !signalB;  
        const btnB =  
document.getElementById('inputB');  
        btnB.classList.toggle('active',  
signalB);  
  
document.getElementById('stateB').innerTe  
xt = signalB ? 'BINARY 1 (ON)' : 'BINARY 0  
(OFF)';  
    }  
}
```

```
    }  
    updateCanvasLogic();  
}
```

```
function updateControls() {  
    const freq =  
document.getElementById('freqSlider').valu  
e;
```

```
document.getElementById('freqVal').innerT  
ext = `${(freq * 0.96).toFixed(1)} GHz`;
```

```
    maxQubitsTarget =  
parseInt(document.getElementById('qubitS  
lider').value);
```

```
document.getElementById('qubitTargetVal')  
.innerText = `${  
{maxQubitsTarget.toLocaleString()}  
Qubits`;  
}
```

```
function updateCanvasLogic() {  
    const wireA =  
document.getElementById('wireA');  
    const wireB =  
document.getElementById('wireB');  
    const wireOut =  
document.getElementById('wireOut');  
    const logicOutput =  
document.getElementById('logicOutput');  
  
    wireA.setAttribute('stroke', signalA ?  
'var(--q-cyan)' : '#334155');  
  
wireB.setAttribute('stroke', signalB ? 'var(--  
q-cyan)' : '#334155');  
  
// XOR Gate Logic Engine  
    const outResult = (signalA ^  
signalB) === 1;  
    wireOut.setAttribute('stroke',
```



```
outResult ? 'var(--q-green)' : '#ef4444');  
    logicOutput.innerText = outResult ?  
'HIGH (1)' : 'LOW (0)';  
    logicOutput.style.color = outResult ?  
'var(--q-green)' : '#ef4444';  
}
```

// QUBIT MEASUREMENT DETECTOR
SIMULATOR (Up to 61,000 Qubits)

```
function updateQubitDetector() {  
    if(currentQubits < maxQubitsTarget)  
{  
        currentQubits +=  
Math.floor(Math.random() * 800) + 200;  
        if(currentQubits >  
maxQubitsTarget) currentQubits =  
maxQubitsTarget;  
    } else if(currentQubits >  
maxQubitsTarget) {  
        currentQubits = maxQubitsTarget;  
    }  
}
```

```
document.getElementById('qubitCounter').i
nnerText = ` $
{currentQubits.toLocaleString()} / $
{maxQubitsTarget.toLocaleString()} `;
    const percentage = (currentQubits /
61000) * 100;

document.getElementById('qubitProgress').
style.width = `${percentage}%`;
    }

setInterval(updateQubitDetector, 150);

// CANVAS OSCILLOSCOPE WAVEFORM
    let scopeAnimId;
    function initOscilloscope() {
        const canvas =
document.getElementById('scopeCanvas');
        const ctx = canvas.getContext('2d');
        canvas.width = canvas.offsetWidth;
```

```
    canvas.height =  
    canvas.offsetHeight;  
  
let step = 0;  
    function draw() {  
        ctx.fillStyle = 'rgba(0, 0, 0, 0.2)';  
        ctx.fillRect(0, 0, canvas.width,  
canvas.height);  
  
        ctx.beginPath();  
        ctx.lineWidth = 2;  
        ctx.strokeStyle = '#10b981';  
  
        for (let x = 0; x < canvas.width; x++) {  
            const y = canvas.height / 2 +  
Math.sin((x + step) * 0.03) * 30 +  
Math.sin((x + step) * 0.08) * 10;  
            if (x === 0) ctx.moveTo(x, y);  
            else ctx.lineTo(x, y);  
        }  
        ctx.stroke();  
    }  
}
```

```
        step += 4;
        scopeAnimId =
requestAnimationFrame(draw);
    }
    if(scopeAnimId)
cancelAnimationFrame(scopeAnimId);
    draw();
}
</script>
</body>
</html>
```